Jack in the Box Simulation

Mira Norman

December 12, 2024

I chose to do the default project.

Analysis

Frames and Rigid Body Transformations

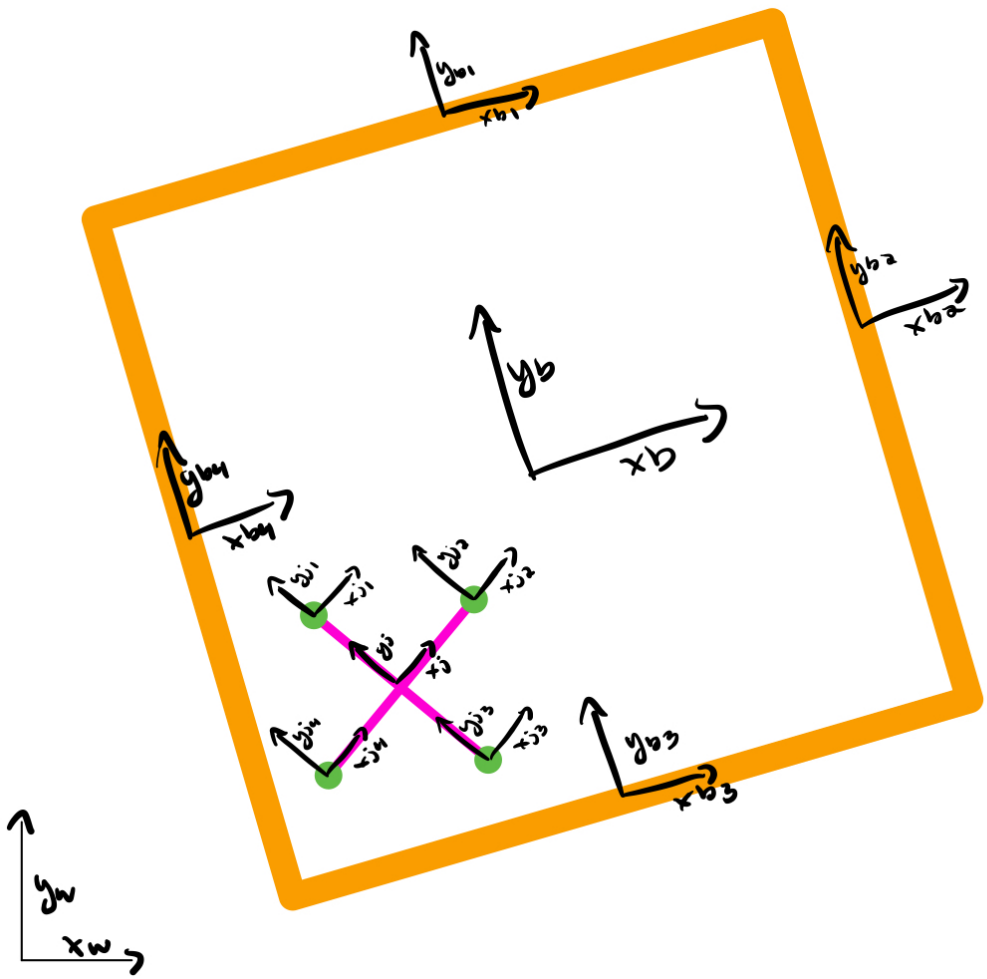

Figure 1: Drawing of Simulation Frames

(Note: due to human error, the drawing may not perfectly represent the system, such as an axis being slightly off center.)

The figure above describes the frames used in calculating all rigid body transformations. Frame w is the world frame, shown by the x and y axes at the bottom left corner. Frame b is the x and y axes that move with the box. Frame b is further divided into four subframes, one for each side of the box. The origin of each axis is in the middle of the length on each side. In the code they are represented as: $g_{wb} = \begin{bmatrix} cos\,(\theta_b) & -sin\,(\theta_b) & 0 & x_b \\ sin\,(\theta_b) & cos\,(\theta_b) & 0 & y_b \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $g_{b1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & \frac{L_b}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $g_{b2} = \begin{bmatrix} 1 & 0 & 0 & \frac{L_b}{2} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $g_{b1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & \frac{-L_b}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, and $g_{b1} = \begin{bmatrix} 1 & 0 & 0 & \frac{-L_b}{2} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$.
Frame j, or the frame around the jack, is evaluated similarly. It is divided into four frames at all four different ends of the jack. Its axes and collinearity share similar traits to the subframes of b. Below are the representations of the j frames. $g_{wj} = \begin{bmatrix} cos\,(\theta_j) & -sin\,(\theta_j) & 0 & x_j \\ sin\,(\theta_j) & cos\,(\theta_j) & 0 & y_j \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$. $g_{b1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & L_j \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $g_{b2} = \begin{bmatrix} 1 & 0 & 0 & L_j \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $g_{b1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -L_j \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, and $g_{b1} = \begin{bmatrix} 1 & 0 & 0 & -L_j \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$.

The following calculations are done below to determine the homogeneous transformations between all frames and subframes. First, the transformations between the frames: Where these final calculations were done to determine

```
gw_b1 = gw_b * gb_1
gw_b2 = gw_b * gb_2
gw_b3 = gw_b * gb_3
gw_b4 = gw_b * gb_4
```

Figure 2: Computing transformations between body subframes and world frame.

```
gw_j1 = gw_j * gj_1
gw_j2 = gw_j * gj_2
gw_j3 = gw_j * gj_3
gw_j4 = gw_j * gj_4
```

Figure 3: Computing transformations between jack subframes and world frame.

the transformations between each subframes of the body and the jack. Where "inv" indicates a matrix is being inverted.

```
gw_b1_j1 = inv(gw_b1) * gw_j1
gw_b1_j2 = inv(gw_b1) * gw_j2
gw_b1_j3 = inv(gw_b1) * gw_j3
gw_b1_j4 = inv(gw_b1) * gw_j4
gw_b2_j1 = inv(gw_b2) * gw_j1
gw_b2_j2 = inv(gw_b2) * gw_j2
gw_b2_j3 = inv(gw_b2) * gw_j3
gw_b2_j4 = inv(gw_b2) * gw_j4
gw_b3_j1 = inv(gw_b3) * gw_j1
gw_b3_j2 = inv(gw_b3) * gw_j2
gw_b3_j3 = inv(gw_b3) * gw_j3
gw_b3_j4 = inv(gw_b3) * gw_j4
gw_b4_j1 = inv(gw_b4) * gw_j1
gw_b4_j2 = inv(gw_b4) * gw_j2
gw_b4_j3 = inv(gw_b4) * gw_j3
gw_b4_j4 = inv(gw_b4) * gw_j4
```

Figure 4: Figure 1: Simulation Frames

External Forces

My external forces in this project are an upward force on the box F_{y_b} to counteract gravity (this is to avoid my box sinking off screen) and a force that causes the box to spin, F_{θ_b}.

I calculated F_{y_b} by multiplying the mass of the box by the gravity constant and 1.10. The 1.10 was implemented because the jack bouncing in the box may account for the box moving down. This is to prevent that from happening.

F_{θ_b} is an arbitrary number selected at will.

Euler-Lagrange Equations

The tried and true formula to find the Euler-Lagrange equations is the following:

$$\frac{d}{dt}\frac{dL}{d\dot{q}} - \frac{dL}{dq} = F$$

.

Where

$$q = \begin{bmatrix} x_b \\ y_b \\ \theta_b \\ x_j \\ y_j \\ \theta_j \end{bmatrix}$$

and F is a 6x1 vector of external forces.

The first step to this is finding the Lagrangian. The Lagrangian here is a measure of the change in kinetic energy and potential energy in the system. In this case, the velocity for any particular frame can be equated to:

$$\hat{V}_{frame} = g_{frame}^{-1}\dot{g}_{frame}$$

.

$$KE = V_{total}^T I_{total} V_{total}$$

It is important to note that I is the inertia frame which in this case is equivalent to $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & J_{frame} \end{bmatrix}$ where J_{frame} is either J_b or J_j and is a predetermined value.

I had to find the individual hatted velocities for the box frame, each subframe of the box, and jack frame. Before I could do other operations, I also had to unhat them. But afterwards, plugging each velocity into the kinetic energy equation above and adding them together found the net kinetic energy.

The potential energy was simpler. It was an average of the four box frames' potential energy due to gravity plus the jack's potential energy due to gravity.

From there, use the equation:

$$L = KE - PE$$

turn the Lagrangian into a matrix, and apply it in terms of q and t so that $\frac{d}{dt}\frac{dL}{d\dot{q}} - \frac{dL}{dq} = F$. Putting in the proper values of F, this equation becomes:

$$\frac{d}{dt}\frac{dL}{d\dot{q}} - \frac{dL}{dq} = \begin{bmatrix} 0 \\ F_{y_b} \\ F_{\theta_b} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

.

From here, the accelerations of each variable in q, or $\ddot{q}$, can be isolated and solved for.

Constraints and Impact

The constraints for movement are directly related to impact. The idea is that the system measures if the ends of the jacks are close enough to the walls of the box that no move movement in one direction can occur and an impact happens. This means the movement of each of the four ends of the jack is is being constrained by each of the four box walls. As a result, all sixteen homogeneous transformations between the body and the jack are the related constraints.

Specifically, whether or not the origin of a jack has made contact with a relevant axis of a wall is what is important. For walls 1 and 3, this is the x-axis. For walls 2 and 4, this is the y-axis.

So the ϕ matrix is the position of each relevant axis in relation to the position of each end of the jack. As put in the program, if the distance between any wall and jack end ≤ 0.35, an impact occurs and updates the equation.

Code Explanation

```
{Derivative(th_b(t), (t, 2)): 0.203045685279188,
 Derivative(th_j(t), (t, 2)): 0.0,
 Derivative(x_b(t), (t, 2)): 0.0,
 Derivative(x_j(t), (t, 2)): 0.0,
 Derivative(y_b(t), (t, 2)): 0.0,
 Derivative(y_j(t), (t, 2)): -9.81000000000000}
```

Figure 5: Solutions to Euler-Lagrange Equations

The program takes initial variables, those being L_b, L_j, m_b, m_j, J_b, J_j, and g. They are the length of a box wall, the length of a jack end, the mass of the box, the mass of the jack, the inertia of the box, the inertia of the jack, and gravity.

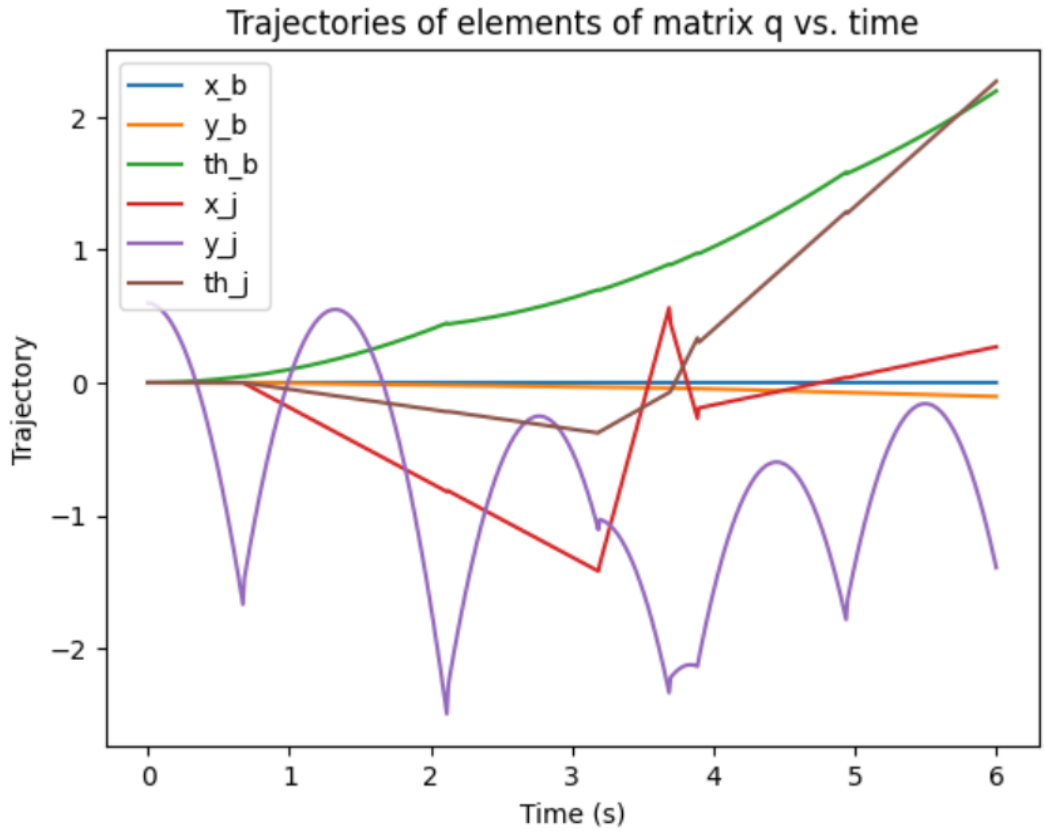

Figure 6: Trajectories vs. Time Plot of q Elements

It also initializes parts of the matrix q as functions, those being x_b, y_b, θ_b, x_j, y_j, and θ_j. It differentiates q by time t to have $\dot{q}$ and $\ddot{q}$. Establish $F = \begin{bmatrix} 0 \\ m_b * g * 4 \\ 6000 \\ 0 \\ 0 \\ 0 \end{bmatrix}$.

The mathematical steps to find the different required sixteen frames was discussed earlier. In finding the transformations between subframes of the box and jack, a helper inverse function was created to make the process smoother.

Inertia matrices were initialized by, and the velocities for each subframe of the box and the jack frame are calculated using the methods already described earlier in this report. "unhat" and inverse helper functions are used in these calculations to streamline the process. With the velocities found, the kinetic energy, potential energy, then Lagrangian is calculated using the processes described earlier.

With the Lagrangian found, the Euler-Lagrange equations are calculated using the mathematical processes described earlier. Then, the terms of $\ddot{q}$ are solved in terms of the Euler-Lagrange equations. Once those solutions are solved using the solve function of sympy, they are lambdified so they can become a function in terms of q and $\dot{q}$.

Using the "x_b_func" and "y_b_func" helper functions to streamline the process, the constraint matrices for each subframe of the box are front, then added together in a big matrix called "phi_ultra_matrix". This matrix is then lambdified for the elements of q and $\dot{q}$.

Impact_condition takes the phi data at any given point of a simulation and a set threshold and determines if any of the phi data has passed a given threshhold. If the threshold was passed, then it returns true. If the threshold was not passed, it returns false. The impact_update function takes the current frame data at a given point that is marked by impact_condition to have an impact occur and update the resulting interaction based on the conditions of the scene. Both of these are used as helper functions to simulate_jack_in_da_box.

The dummy variables for conditions τ^- and τ^+ allow the Hamiltonian to be observed and to use the difference

between the conditions at the times to evaluate the impact equations.

The dyn function inputs a function and returns with an array that includes the q data and the acceleration data.

The simulate_jack_in_da_box function is what pulls everything together and actually simulates the dynamics of the system. It calculates the trajectory based on the constraints placed. It uses the helper functions already established to see if there is an impact and changes course accordingly if there is a change, using the impact_update function to figure out course. It returns an array of trajectories that can be used for further calculations or visual aids.

The animate_in_da_box function goes through the mathematical frame and subframe calculations to animate where they are in space. It takes in an array of trajectories, the length of a box wall, the length of a jack piece, and the time in seconds it will animate for. It goes through the motions of the stored information about from the change in q as well as the change in acceleration. It calculates the movement of each frame and visualizes it frame by frame.

Finally, using the dyn helper function, the state data stated earlier, a tspan of [0,6], and a change in time of 0.01, and adding the integrate function as a helper function, the trajectory traj is simulated and output as an array.

x_val is calculated using the span of tspan and the number of instances in trajectory. Then, x_val is plotted with all the values of q in the traj output with the matlab plotter.

It is also animated using the animate_in_da_box function.

Code

```
# import necessary packages
import sympy as sym
import numpy as np
import math
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

```
# helper functions
def unhat(hat):
  nohat = sym.Matrix([hat[0,3], hat[1,3], hat[2,3], hat[2,1], hat[0,2], hat[1,0]])
  return nohat

def inv(inver):
    R = inver[0:2, 0:2]
    p = inver[0:2, 3]
    return sym.Matrix([[R.T, sym.Matrix([0,0]), -R.T*p],
                       [0, 0, 1, 0],
                       [0, 0, 0, 1]])

def x_b_func(g):
  return g[0,3]

def y_b_func(g):
  return g[1,3]

def integrate(f, xt, dt):
    # Perform numerical integration using the 4th-order Runge-Kutta method (RK4).
    k1 = dt * f(xt)
    k2 = dt * f(xt+k1/2)
    k3 = dt * f(xt+k2/2)
    k4 = dt * f(xt+k3)
    new_xt = xt + (1/6) * (k1+2.0*k2+2.0*k3+k4)
    return new_xt
```

```
# initialize variables with constant values
L_b = 6
L_j = 1
m_b = 800
m_j = 1
J_b = 150
J_j = 15
g = 9.81
```

```python
# dictionary for substitutions for our constants
numerical_values = {
    L_b: 6,
    L_j: 1,
    m_b: 800,
    m_j:1,
    J_b: 150,
    J_j: 15,
    g:9.81
}

# initial conditions for the system, used in the simulate and dynamics functions
state = np.array([0, 0, 0, 0, 0.6, 0, 0, 0, 0, 0, 0, 0])

# initialize functions in q matrix
t = sym.symbols('t')
x_b = sym.Function('x_b')(t)
y_b = sym.Function('y_b')(t)
th_b = sym.Function('th_b')(t)
x_j = sym.Function('x_j')(t)
y_j = sym.Function('y_j')(t)
th_j = sym.Function('th_j')(t)

# external forces
F_y = m_b*g*4
F_th = 6000
F = sym.Matrix([0,F_y, F_th, 0,0,0])

# q matrix
q = sym.Matrix([x_b, y_b, th_b, x_j, y_j, th_j])
qdot = q.diff(t)
qddot = qdot.diff(t)
```

```python
# define homogenous transformations
# transformation with reference to world frame and box frame
gw_b = sym.Matrix([[sym.cos(th_b),-sym.sin(th_b),0,x_b],
                   [sym.sin(th_b),sym.cos(th_b),0,y_b],
                   [0,0,1,0],
                   [0,0,0,1]])
# transformation with reference to world frame and jack frame
gw_j = sym.Matrix([[sym.cos(th_j),-sym.sin(th_j),0,x_j],
                   [sym.sin(th_j),sym.cos(th_j),0,y_j],
                   [0,0,1,0],
                   [0,0,0,1]])
# transformations for box frame for each box wall
gb_1 = sym.Matrix([[1,0,0,0],
                    [0,1,0,L_b/2],
                    [0,0,1,0],
                    [0,0,0,1]])
gb_2 = sym.Matrix([[1,0,0,L_b/2],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])
gb_3 = sym.Matrix([[1,0,0,0],
                    [0,1,0,-L_b/2],
                    [0,0,1,0],
                    [0,0,0,1]])
gb_4 = sym.Matrix([[1,0,0,-L_b/2],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])

# individual box wall frame and world frame transformations
gw_b1 = gw_b * gb_1
gw_b2 = gw_b * gb_2
gw_b3 = gw_b * gb_3
gw_b4 = gw_b * gb_4

# individual jack frame for each jack part
gj_1 = sym.Matrix([[1,0,0,0],
```

```
                    [0,1,0,L_j],
                    [0,0,1,0],
                    [0,0,0,1]])
gj_2 = sym.Matrix([[1,0,0,L_j],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])
gj_3 = sym.Matrix([[1,0,0,0],
                    [0,1,0,-L_j],
                    [0,0,1,0],
                    [0,0,0,1]])
gj_4 = sym.Matrix([[1,0,0,-L_j],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])

# individual jack frames and world frame transformations
gw_j1 = gw_j * gj_1
gw_j2 = gw_j * gj_2
gw_j3 = gw_j * gj_3
gw_j4 = gw_j * gj_4

# transformations between each individual box frame and each individual jack frame
gw_b1_j1 = inv(gw_b1) * gw_j1
gw_b1_j2 = inv(gw_b1) * gw_j2
gw_b1_j3 = inv(gw_b1) * gw_j3
gw_b1_j4 = inv(gw_b1) * gw_j4
gw_b2_j1 = inv(gw_b2) * gw_j1
gw_b2_j2 = inv(gw_b2) * gw_j2
gw_b2_j3 = inv(gw_b2) * gw_j3
gw_b2_j4 = inv(gw_b2) * gw_j4
gw_b3_j1 = inv(gw_b3) * gw_j1
gw_b3_j2 = inv(gw_b3) * gw_j2
gw_b3_j3 = inv(gw_b3) * gw_j3
gw_b3_j4 = inv(gw_b3) * gw_j4
gw_b4_j1 = inv(gw_b4) * gw_j1
gw_b4_j2 = inv(gw_b4) * gw_j2
gw_b4_j3 = inv(gw_b4) * gw_j3
gw_b4_j4 = inv(gw_b4) * gw_j4
```

```
# intertia matrices for body frame and jack frame
I_b = sym.Matrix([[m_b,0,0,0,0,0],
                  [0,m_b,0,0,0,0],
                  [0,0,m_b,0,0,0],
                  [0,0,0,0,0,0],
                  [0,0,0,0,0,0],
                  [0,0,0,0,0,J_b]])
I_j = sym.Matrix([[m_j*4,0,0,0,0,0],
                  [0,m_j*4,0,0,0,0],
                  [0,0,m_j*4,0,0,0],
                  [0,0,0,0,0,0],
                  [0,0,0,0,0,0],
                  [0,0,0,0,0,J_j]])
```

```
# calculate individual velocities
v_b = unhat(inv(gw_b)*gw_b.diff(t))
v_b1 = unhat(inv(gw_b1)*gw_b1.diff(t))
v_b2 = unhat(inv(gw_b2)*gw_b2.diff(t))
v_b3 = unhat(inv(gw_b3)*gw_b3.diff(t))
v_b4 = unhat(inv(gw_b4)*gw_b4.diff(t))
v_j = unhat(inv(gw_j)*gw_j.diff(t))

# calculate Lagrangian using the sum of the velocities in ke
ke = sym.simplify(0.5*(((v_b.T*I_b*v_b))+((v_b1.T*I_b*v_b1))+((v_b2.T*I_b*v_b2))+((v_b3.T*I_b*v_b3
                                                    ))+((v_b4.T*I_b*v_b4))+((v_j.T*I_j*v_j))))
pe = sym.simplify((4*g*((m_b/4*(y_b_func(gw_b1)+y_b_func(gw_b2)+y_b_func(gw_b3)+y_b_func(gw_b4)))
                                                    + m_j*y_j)))
L = sym.simplify(ke[0]-pe)
L = L.subs(numerical_values)
```

```
# calculate and solve Euler-Lagrange equations by finding d_dLdqdot_dt - dLdq = F
L_mat = sym.Matrix([L])
dLdq = L_mat.jacobian(q)
dLdqdot = L_mat.jacobian(qdot)
d_dLdqdot_dt = dLdqdot.diff(t)
el = sym.simplify(d_dLdqdot_dt - dLdq)
el_trans = el.T
el_eqn = sym.Eq(el_trans, F)
el_sol = sym.solve(el_eqn, qddot)
display(el_sol)

# lambdify solutions
sol_lambda = {}
for i in el_sol:
    sol_lambda[i] = sym.lambdify([[q[0], q[1], q[2], q[3], q[4], q[5], qdot[0], qdot[1], qdot[2],
                                                    qdot[3], qdot[4], qdot[5]]],el_sol[i])
```

```
# define constraint matrix, first finding individual contraint matrices for each box wall and
                                                    adding them together
phi_b_1 = [-y_b_func(gw_b1_j1), -y_b_func(gw_b1_j2), -y_b_func(gw_b1_j3), -y_b_func(gw_b1_j4)]
phi_b_2 = [x_b_func(gw_b2_j1), x_b_func(gw_b2_j2), x_b_func(gw_b2_j3), x_b_func(gw_b2_j4)]
phi_b_3 = [y_b_func(gw_b3_j1), y_b_func(gw_b3_j2), y_b_func(gw_b3_j3), y_b_func(gw_b3_j4)]
phi_b_4 = [-x_b_func(gw_b4_j1), -x_b_func(gw_b4_j2), -x_b_func(gw_b4_j3), -x_b_func(gw_b4_j4)]
phi_ultra_mat = sym.Matrix(phi_b_1 + phi_b_2 + phi_b_3 + phi_b_4)

# lambdify the constraint equations using the phi matrix
lamb = sym.symbols('lambda')
phi_func = sym.lambdify([q[0], q[1], q[2], q[3], q[4], q[5], qdot[0], qdot[1], qdot[2], qdot[3],
                                                    qdot[4], qdot[5]], phi_ultra_mat)
impact_eqns = []
```

```
# updates impact equations at given i in a matrix
def impact_update(mat, i):
  impact_eqns[i] = impact_eqns[i].subs({x_b_min: mat[0], y_b_min: mat[1],
                                                  th_b_min: mat[2], x_j_min: mat[3],
                                                  y_j_min: mat[4], th_j_min: mat[5],
                                                  xdot_b_min: mat[6], ydot_b_min: mat[7],
                                                  thdot_b_min: mat[8], xdot_j_min: mat[9],
                                                  ydot_j_min: mat[10], thdot_j_min: mat[11]})
  sols = sym.solve(impact_eqns[i], [xdot_b_pl, ydot_b_pl, thdot_b_pl,
                                          xdot_j_pl, ydot_j_pl, thdot_j_pl,
                                          lamb],dict=True)
  lam_nz = {lamb: 0}
  for sol in sols:
    if np.abs(sol[lamb]) >= np.abs(lam_nz[lamb]):
     lam_nz = sol
  return lam_nz

# determines if the impact condition threshold is reached for an impact to occur, helper to
                                                    simulate
def impact_condition(mat, thresh):
  phiq = phi_func(*mat)
  for i,phi in enumerate(phiq):
    if -thresh < phi and thresh >phi:
      return True, i
  return False, i
```

```
# initialize dummy variables for tau- and tau+
x_b_min, y_b_min, th_b_min, x_j_min, y_j_min, th_j_min = sym.symbols('x-, y_b-, th_b-, x_j-, y_j-,
                                                    th_j-')
xdot_b_min, ydot_b_min, thdot_b_min, xdot_j_min, ydot_j_min, thdot_j_min = sym.symbols('xdot_b-,
                                                    ydot_b-, thdot_b-, xdot_j-, ydot_j-, thdot_j-')
xdot_b_pl, ydot_b_pl, thdot_b_pl, xdot_j_pl, ydot_j_pl, thdot_j_pl = sym.symbols('xdot_b+, ydot_b
                                                    +, thdot_b+, xdot_j+, ydot_j+, thdot_j+')

# initialize dummy dictionaries
min_dict = {
    q[0]: x_b_min, q[1]: y_b_min, q[2]: th_b_min, q[3]:x_j_min, q[4]:y_j_min, q[5]:th_j_min,
    qdot[0]: xdot_b_min, qdot[1]: ydot_b_min, qdot[2]: thdot_b_min, qdot[3]:xdot_j_min, qdot[4]:
                                                    ydot_j_min, qdot[5]:thdot_j_min
```

```
    }
pl_dict = {
    q[0]: x_b_min, q[1]: y_b_min, q[2]: th_b_min, q[3]:x_j_min, q[4]:y_j_min, q[5]:th_j_min,
    qdot[0]: xdot_b_pl, qdot[1]: ydot_b_pl, qdot[2]: thdot_b_pl, qdot[3]:xdot_j_pl, qdot[4]:
                                                        ydot_j_pl, qdot[5]:thdot_j_pl
}

# evaluate Hamiltonian and use minus and plus dictionaries to calculate impact equations
H = (dLdqdot*qdot)[0] - L
H_min = H.subs(min_dict)
H_pl = H.subs(pl_dict)

dphidq = phi_ultra_mat.jacobian(q)
dphidq_min = dphidq.subs(min_dict)
dphidq_pl = dphidq.subs(pl_dict)

dLdqdot_min = dLdqdot.subs(min_dict)
dLdqdot_pl = dLdqdot.subs(pl_dict)

# find the left and right hand parts of the impact equations for each instance and save them in
                                                    impact_eqns.append
for i in range(dphidq_min.shape[0]):
    l = sym.Matrix([
        dLdqdot_pl[0] - dLdqdot_min[0], dLdqdot_pl[1] - dLdqdot_min[1],
        dLdqdot_pl[2] - dLdqdot_min[2], dLdqdot_pl[3] - dLdqdot_min[3],
        dLdqdot_pl[4] - dLdqdot_min[4], dLdqdot_pl[5] - dLdqdot_min[5],
        H_pl - H_min
    ])
    r = sym.Matrix([
        lamb*dphidq_min[i, 0], lamb*dphidq_min[i, 1], lamb*dphidq_min[i, 2],
        lamb*dphidq_min[i, 3], lamb*dphidq_min[i, 4], lamb*dphidq_min[i, 5], 0
    ])

    # Create the equation and add it to the list of impact equations
    i_eq = sym.Eq(l, r)
    impact_eqns.append(i_eq)
```

```
def dyn(s):
    # returns dynamic array with movement data
    return np.array([s[6], s[7], s[8], s[9], s[10], s[11],
                     sol_lambda[qddot[0]](s),
                     sol_lambda[qddot[1]](s),
                     sol_lambda[qddot[2]](s),
                     sol_lambda[qddot[3]](s),
                     sol_lambda[qddot[4]](s),
                     sol_lambda[qddot[5]](s)])

# simulates the jack and the box movement data
def simulate_jack_in_da_box(f, x0, tspan, dt, integrate):
    thresh = .35 #threshhold for impact to occur
    N = int((max(tspan) - min(tspan)) / dt)
    x = np.copy(x0)
    tvec = np.linspace(min(tspan), max(tspan), N)
    imtraj = np.zeros((len(x0), N))

    # checks if an impact has occured at any instance and acts accordingly if one has occured
    for i in range(N):
        impact, index = impact_condition(x, thresh)
        # if an impact has occured based on thresh, the following data is taken and the system
                                                        reacts with an impact
        if impact:
            x = np.copy(imtraj[:, i - 3])
            lam_nz = impact_update(x, index)
            imtraj[:, i] = integrate(f,
                                     [x[0], x[1], x[2], x[3], x[4], x[5],
                                     lam_nz[xdot_b_pl],lam_nz[ydot_b_pl],
                                     lam_nz[thdot_b_pl],lam_nz[xdot_j_pl],
                                     lam_nz[ydot_j_pl],lam_nz[thdot_j_pl]], dt)
            x = np.copy(imtraj[:, i])  # Update state
        # if an impact does not occur, the system keeps going as before
        else:
```

```
            imtraj[:, i] = integrate(f, x, dt)
            x = np.copy(imtraj[:, i])
    return imtraj
```

```
# animation function
def animate_jack_in_da_box(theta_array,L_box=6,L_jack=1,T=6):
    ################################
    # Imports required for animation.
    from plotly.offline import init_notebook_mode, iplot
    from IPython.display import display, HTML
    import plotly.graph_objects as go

    ########################
    # Browser configuration.
    def configure_plotly_browser_state():
        import IPython
        display(IPython.core.display.HTML('''
            <script src="/static/components/requirejs/require.js"></script>
            <script>
              requirejs.config({
                paths: {
                  base: '/static/base',
                  plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
                },
              });
            </script>
            '''))
    configure_plotly_browser_state()
    init_notebook_mode(connected=False)

    ################################################
    # below are the parts of q put into an array
    xx_box = theta_array[0]
    yy_box = theta_array[1]
    theta_box_array = theta_array[2]
    xx_jack = theta_array[3]
    yy_jack = theta_array[4]
    theta_jack_array = theta_array[5]

    # N = the number of steps
    N = len(theta_array[0])
    ################################################
    # Initialize arrays for transformation
    x_axis = np.array([0.3, 0.0])
    y_axis = np.array([0.0, 0.3])

    # frame data for all frames
    frame_b1_x_axis = np.zeros((2,N))
    frame_b1_y_axis = np.zeros((2,N))
    frame_b2_x_axis = np.zeros((2,N))
    frame_b2_y_axis = np.zeros((2,N))
    frame_b3_x_axis = np.zeros((2,N))
    frame_b3_y_axis = np.zeros((2,N))
    frame_b4_x_axis = np.zeros((2,N))
    frame_b4_y_axis = np.zeros((2,N))
    frame_j1_x_axis = np.zeros((2,N))
    frame_j1_y_axis = np.zeros((2,N))
    frame_j2_x_axis = np.zeros((2,N))
    frame_j2_y_axis = np.zeros((2,N))
    frame_j3_x_axis = np.zeros((2,N))
    frame_j3_y_axis = np.zeros((2,N))
    frame_j4_x_axis = np.zeros((2,N))
    frame_j4_y_axis = np.zeros((2,N))

    # keep data for box frame
    box_top_left = np.zeros((2,N))
    box_top_right = np.zeros((2,N))
    box_bottom_right = np.zeros((2,N))
    box_bottom_left = np.zeros((2,N))

    # keep data for jack frame
```

```
jack_top = np.zeros((2,N))
jack_right = np.zeros((2,N))
jack_bottom = np.zeros((2,N))
jack_left = np.zeros((2,N))

for i in range(N):
    # transformation world-body frame reference
    twb = np.array([[np.cos(theta_box_array[i]), -np.sin(theta_box_array[i]), 0, xx_box[i]],
                    [np.sin(theta_box_array[i]), np.cos(theta_box_array[i]), 0, yy_box[i]],
                    [0,0,1,0],
                    [0,0,0,1]])
    # transformation world-jack frame reference
    twj = np.array([[np.cos(theta_jack_array[i]), -np.sin(theta_jack_array[i]), 0, xx_jack[i]]
                                                                                ,
                    [np.sin(theta_jack_array[i]), np.cos(theta_jack_array[i]), 0, yy_jack[i]]
                                                                                            ,
                    [0,0,1,0],
                    [0,0,0,1]])
    # sub frame transformations for box parts
    tbb1 = np.array([[1,0,0,0],
                    [0,1,0,L_box/2],
                    [0,0,1,0],
                    [0,0,0,1]])
    tbb2 = np.array([[1,0,0,L_box/2],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])
    tbb3 = np.array([[1,0,0,0],
                    [0,1,0,-L_box/2],
                    [0,0,1,0],
                    [0,0,0,1]])
    tbb4 = np.array([[1,0,0,-L_box/2],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])
    # sub frame transformations for jack parts
    tjj1 = np.array([[1,0,0,0],
                    [0,1,0,L_jack],
                    [0,0,1,0],
                    [0,0,0,1]])
    tjj2 = np.array([[1,0,0,L_jack],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])
    tjj3 = np.array([[1,0,0,0],
                    [0,1,0,-L_jack],
                    [0,0,1,0],
                    [0,0,0,1]])
    tjj4 = np.array([[1,0,0,-L_jack],
                    [0,1,0,0],
                    [0,0,1,0],
                    [0,0,0,1]])

    # middle step - applying world-part frames to subframes
    twb1 = twb @ tbb1
    twb2 = twb @ tbb2
    twb3 = twb @ tbb3
    twb4 = twb @ tbb4

    twj1 = twj @ tjj1
    twj2 = twj @ tjj2
    twj3 = twj @ tjj3
    twj4 = twj @ tjj4

    # calculating frame data for each instance
    frame_b1_x_axis[:,i] = twb1.dot([L_box/2, L_box/2, 0, 1])[0:2]
    frame_b1_y_axis[:,i] = twb1.dot([L_box/2, L_box/2, 0, 1])[0:2]
    frame_b2_x_axis[:,i] = twb2.dot([L_box/2, -L_box/2, 0, 1])[0:2]
    frame_b2_y_axis[:,i] = twb2.dot([L_box/2, -L_box/2, 0, 1])[0:2]
    frame_b3_x_axis[:,i] = twb3.dot([-L_box/2, -L_box/2, 0, 1])[0:2]
    frame_b3_y_axis[:,i] = twb3.dot([-L_box/2, -L_box/2, 0, 1])[0:2]
```

```
    frame_b4_x_axis[:,i] = twb4.dot([-L_box/2, L_box/2, 0, 1])[0:2]
    frame_b4_y_axis[:,i] = twb4.dot([-L_box/2, L_box/2, 0, 1])[0:2]

    frame_j1_x_axis[:,i] = twj1.dot([L_jack, L_jack, 0, 1])[0:2]
    frame_j1_y_axis[:,i] = twj1.dot([L_jack, L_jack, 0, 1])[0:2]
    frame_j2_x_axis[:,i] = twj2.dot([L_jack, -L_jack, 0, 1])[0:2]
    frame_j2_y_axis[:,i] = twj2.dot([L_jack, -L_jack, 0, 1])[0:2]
    frame_j3_x_axis[:,i] = twj3.dot([-L_jack, -L_jack, 0, 1])[0:2]
    frame_j3_y_axis[:,i] = twj3.dot([-L_jack, -L_jack, 0, 1])[0:2]
    frame_j4_x_axis[:,i] = twj4.dot([-L_jack, L_jack, 0, 1])[0:2]
    frame_j4_y_axis[:,i] = twj4.dot([-L_jack, L_jack, 0, 1])[0:2]

    # box data at the instance from the beginning
    box_top_left[:,i] = twb1.dot([-L_box/2, 0, 0, 1])[0:2]
    box_top_right[:,i] = twb2.dot([0, L_box/2, 0, 1])[0:2]
    box_bottom_right[:,i] = twb3.dot([L_box/2, 0, 0, 1])[0:2]
    box_bottom_left[:,i] = twb4.dot([0, -L_box/2, 0, 1])[0:2]

    # jack data at the instance from the beginning
    jack_top[:,i] = twj1.dot([0, 0, 0, 1])[0:2]
    jack_right[:,i] = twj2.dot([0, 0, 0, 1])[0:2]
    jack_bottom[:,i] = twj3.dot([0, 0, 0, 1])[0:2]
    jack_left[:,i] = twj4.dot([0, 0, 0, 1])[0:2]

####################################
# Define the plot limits for the animation.
xm = -10
xM = 10
ym = -10
yM = 10

############################
# Defining data dictionary for the components to animate.
data=[
    dict(name="Box"),
    dict(name="Jack Leg 1"),
    dict(name="Jack Leg 2"),
    dict(name="Jack Ends")
    ]

################################
# Preparing simulation layout.
layout=dict(autosize=False, width=750, height=750,
            xaxis=dict(range=[xm, xM], autorange=False, zeroline=False,dtick=1),
            yaxis=dict(range=[ym, yM], autorange=False, zeroline=False,scaleanchor = "x",dtick
                                                                =1),
            title='Jack in a Box Simulation',
            hovermode='closest',
            updatemenus= [{'type': 'buttons',
                           'buttons': [{'label': 'Play','method': 'animate',
                                        'args': [None, {'frame': {'duration': T, 'redraw':
                                                                                          False
                                                                                          }}]},
                                       {'args': [[None], {'frame': {'duration': T, 'redraw':
                                                                                         False}
                                                                                         , '
                                                                                         mode':
                                                                                          '
                                                                                         immediate
                                                                                         ',
                                        'transition': {'duration': 0}}],'label': 'Pause','
                                                                                          method
                                                                                          ': '
                                                                                          animate
                                                                                          '}
                                      ]
                          }]
           )

#########################################
# Defining the frames of the simulation.
```

```
    frames=[dict(data=[
                        #################### BOX LINES ############################
                        dict(x = [box_top_left[0][k], box_top_right[0][k], box_bottom_right[0][k],
                                                                                box_bottom_left[0][k],
                                                                                box_top_left[0][k]],
                             y = [box_top_left[1][k], box_top_right[1][k], box_bottom_right[1][k],
                                                                                    box_bottom_left[1][k],
                                                                                     box_top_left[1][k]],
                             mode='lines',
                             line=dict(color='orange', width=6)
                             ),
                        #################### JACK POINT LINES ############################
                        dict(x=[jack_top[0][k], jack_bottom[0][k]],
                             y=[jack_top[1][k], jack_bottom[1][k]],
                             mode='lines',
                             line=dict(color='pink', width=1),),
                        dict(x=[jack_left[0][k], jack_right[0][k]],
                             y=[jack_left[1][k], jack_right[1][k]],
                             mode='lines',
                             line=dict(color='pink', width=1),),
                        #################### JACK POINT MASSES ############################
                        go.Scatter(
                              x=[jack_top[0][k], jack_right[0][k], jack_bottom[0][k], jack_left[0][k
                                                                                        ]],
                              y=[jack_top[1][k], jack_right[1][k], jack_bottom[1][k], jack_left[1][k
                                                                                        ]],
                              mode="markers",
                              marker=dict(color="green", size=8),
                        ),
                        ]) for k in range(N)]
    ########################################
    # plot data
    figure1=dict(data=data, layout=layout, frames=frames)
    iplot(figure1)
```

```
# Run the simulation using the new function
traj = simulate_jack_in_da_box(dyn, state, [0, 6], 0.01, integrate)
```

```
# plot the trajectories of the different parts of q
x_val = np.linspace(0,6,len(traj[0]))
plt.plot(x_val, traj[0])
plt.plot(x_val, traj[1])
plt.plot(x_val, traj[2])
plt.plot(x_val, traj[3])
plt.plot(x_val, traj[4])
plt.plot(x_val, traj[5])
plt.title('Trajectories of elements of matrix q vs. time')
plt.xlabel('Time (s)')
plt.ylabel('Trajectory')
plt.legend(['x_b', 'y_b', 'th_b', 'x_j', 'y_j', 'th_j'])
```

```
# animate the system
animate_jack_in_da_box(traj)
```